Universidade do Vale do Itajaí - UNIVALI

Disciplina: Sistemas Operacionais
Avaliação M1 – IPC, Threads e Paralelismo
Trabalho: Emulador de Tradução de Endereços Virtuais para Físicos

Aluno(s): Eduardo Fabeni Johannsen e Kauan David Leal

Professor: Michael Douglas Cabral Alves

Link github:
https://github.com/EduardoFabeniJohannsen/Ciencia_da_Computacao_UNIVALI/tree/main/Sistemas_Operacionais-SEG/Atividade%20Sistemas%20Operacionais

1. Introdução

Este trabalho apresenta um estudo teórico e prático sobre técnicas de gerenciamento de memória utilizadas por sistemas operacionais para tradução de endereços virtuais em endereços físicos. Foram analisadas três abordagens: Paginação Hierárquica, Tabelas de Página com Hash e Tabelas de Páginas Invertidas. Também foi desenvolvido um emulador em linguagem C para simular inserção e busca de páginas.

2. Parte 1 - Pesquisa Teórica

2.1 Paginação Hierárquica

- A paginação hierárquica organiza a tabela de páginas em múltiplos níveis. Em vez de manter uma única tabela grande, o sistema divide essa estrutura em tabelas menores.
- As tabelas secundárias são criadas somente quando há necessidade, ou seja, quando determinada região da memória virtual passa a ser utilizada pelo processo.
- Essa abordagem reduz o desperdício de memória, especialmente em processos que utilizam poucos blocos ou regiões isoladas do espaço virtual.
- Como desvantagem, a tradução de endereços pode exigir mais de um acesso às tabelas até localizar o frame físico correspondente.

2.2 Tabelas de Página com Hash

- As tabelas de página com hash utilizam uma função hash para localizar rapidamente a página lógica desejada.
- Ao receber o número da página virtual, a função hash calcula uma posição provável dentro da tabela, permitindo buscas rápidas.
- Essa técnica é bastante útil em arquiteturas de 64 bits, onde o espaço de endereçamento virtual é muito grande.
- Quando duas páginas geram o mesmo índice, ocorre uma colisão. Nesses casos, normalmente são utilizadas listas encadeadas para armazenar múltiplas entradas na mesma posição.
- Quando a tabela está bem dimensionada e há poucas colisões, o tempo médio de busca tende a ser próximo de O(1).

2.3 Tabelas de Páginas Invertidas

- A tabela de páginas invertida utiliza uma lógica diferente das demais estruturas.
- Em vez de possuir entradas para páginas virtuais, ela possui uma única tabela global baseada na quantidade de frames físicos existentes na memória RAM.
- Cada entrada informa qual página virtual está ocupando determinado frame físico.
- Sua principal vantagem é o menor consumo de memória, pois o tamanho da estrutura depende da memória física instalada e não do espaço virtual dos processos.
- Como desvantagem, a busca pode ser mais lenta caso seja feita de forma sequencial, exigindo apoio de hardware para melhor desempenho.

3. Parte 2 - Implementação Computacional

Foi desenvolvido um programa em linguagem C capaz de simular as três estruturas pesquisadas.

O sistema implementado permite:

- Inserir mapeamentos entre Página Lógica → Frame Físico;
- Buscar páginas previamente cadastradas;
- Retornar Page Fault quando a página não estiver mapeada.

O código-fonte foi disponibilizado separadamente no repositório GitHub informado.

4. Parte 3 - Análise e Conclusões

4.1 Paginação Hierárquica

Economiza memória ao criar tabelas secundárias apenas quando necessário. Em comparação à tabela simples, evita alocar entradas nunca utilizadas.

4.2 Tabela Hash

Se a tabela hash for pequena, aumentam as colisões e o tempo de busca cresce. O sistema operacional precisa equilibrar tamanho da tabela e desempenho.

4.3 Tabela Invertida

A busca sequencial possui pior caso O(n), onde n é a quantidade de frames físicos. Nos sistemas reais, a TLB é essencial para acelerar as traduções.

4.4 Ranking Comparativo

Uso de Memória (melhor para pior):
1. Tabela Invertida -> utiliza uma entrada por frame físico, sendo a mais econômica.
2. Paginação Hierárquica -> aloca tabelas apenas quando necessário.
3. Tabela Simples / Hash mal dimensionada ->Hash mal dimensionada – pode desperdiçar memória com estruturas grandes ou mal ajustadas

Velocidade de Busca (melhor para pior):
1. Hash bem dimensionada -> acesso rápido, próximo de O(1) em média.
2. Paginação Hierárquica -> eficiente, porém exige múltiplos acessos entre níveis.
3. Tabela Invertida sequencial -> necessita percorrer frames até localizar a página.

5. Conclusão Final

Não existe solução perfeita para todos os casos. O gerenciamento de memória envolve trade-off entre consumo de memória e velocidade de acesso. Cada sistema operacional escolhe a estratégia mais adequada conforme hardware e carga de trabalho.